Emotional Prompting 1:-

My resume isn't just a piece of paper—it's the key to unlocking your next career opportunity.

A poorly structured resume can cost my dream job with Amazon. Recruiters make hiring decisions in seconds, and my resume must instantly communicate my skill and value.

Generate an ATS-friendly Selenium Test Automation Engineer resume that highlights technical skills, achievements, and measurable impact, ensuring maximum hiring potential.

[YOUR NAME]
📍 Chennai, India | 📧 your.email@example.com | 📞 +91-XXXXXXXXXX | 🔗 LinkedIn | 🔗 GitHub

---

💻 PROFESSIONAL SUMMARY

Results-driven **Selenium Test Automation Engineer** with X+ years of experience in designing, developing, and maintaining scalable test automation frameworks. Proven ability to improve test efficiency, reduce defects, and accelerate release cycles. Skilled in delivering high-quality software through robust automation strategies aligned with Agile and DevOps practices.

🛠 TECHNICAL SKILLS

- **Automation Tools:** Selenium WebDriver, TestNG, JUnit, Cucumber (BDD)
- **Programming Languages:** Java, Python
- **Frameworks:** Page Object Model (POM), Data-Driven, Hybrid Frameworks
- **CI/CD Tools:** Jenkins, GitHub Actions
- **Version Control:** Git, GitHub, Bitbucket
- **API Testing:** REST Assured, Postman
- **Build Tools:** Maven, Gradle

- **Defect Tracking:** JIRA, Bugzilla
- **Databases:** MySQL, SQL Server
- **Cloud & DevOps:** Docker, AWS (basic)
- **Other:** Agile/Scrum, Test Strategy, Cross-browser Testing

---

💼 PROFESSIONAL EXPERIENCE

Selenium Test Automation Engineer

[Current/Previous Company Name] | [Location] | [Month Year – Present]

- Designed and implemented **scalable Selenium automation framework** from scratch, reducing manual testing effort by **60%**
- Automated **200+ test cases**, improving regression testing speed by **70%**
- Integrated automation suite with **Jenkins CI/CD pipeline**, enabling nightly test runs and faster feedback loops
- Collaborated with developers and QA teams in Agile sprints to ensure **early defect detection**, reducing production defects by **40%**
- Developed reusable utilities and libraries, improving code maintainability and reducing duplication by **30%**
- Performed cross-browser testing using Selenium Grid, ensuring compatibility across Chrome, Firefox, and Edge
- Logged and tracked defects using JIRA, improving bug resolution turnaround time by **25%**

QA Automation Engineer

[Previous Company Name] | [Location] | [Month Year – Month Year]

- Built and maintained **Hybrid Test Automation Framework**, improving execution efficiency by **50%**
- Automated API test cases using REST Assured, reducing backend defects by **35%**
- Conducted data-driven testing, increasing test coverage by **45%**
- Participated in sprint planning, test case design, and daily stand-ups in Agile environment

- Reduced regression cycle time from **5 days to 2 days** through automation

---

📊 KEY ACHIEVEMENTS

- 🚀 Reduced overall testing cycle time by **50%+** through automation
- 🏆 Increased defect detection rate by **40% before production release**
- ⚡ Improved CI/CD pipeline efficiency, enabling **faster deployments**
- ✅ Delivered high-quality releases with **minimal post-production issues**

🎓 EDUCATION

Bachelor of Engineering / Technology in [Your Field]
[University Name] | [Year]

📜 CERTIFICATIONS (Optional)

- Selenium Automation Certification
- ISTQB Certified Tester (if applicable)

🔍 PROJECTS

E-commerce Web Application Automation

- Automated end-to-end test scenarios including login, search, cart, and checkout flows
- Implemented Page Object Model (POM) design pattern for maintainability
- Integrated with Jenkins for continuous testing

▯ ADDITIONAL INFORMATION

- Strong problem-solving and debugging skills
- Excellent communication and teamwork abilities
- Quick learner with adaptability to new tools and technologies

---

✅ **ATS OPTIMIZATION KEYWORDS**

Selenium, Test Automation, Java, TestNG, JUnit, Cucumber, POM, CI/CD, Jenkins, REST Assured, API Testing, Agile, Regression Testing, Automation Framework, QA Engineer

---

Emotional Prompting 2:-

I — INSTRUCTION Create Selenium Java + TestNG automation for the steps below. Generate Testcases for Login Generate Testcases for all scenarios with SSO and MFA Treat every [P0] step as critical. Use Page Object Model, WebDriverWait only (no Thread.sleep), stable locators, and real business assertions.

C — CONTEXT If these [P0] steps fail in production and the tests did not catch it, I will be blamed. There will be questions about why the automation missed it, why it was flaky, and why there was no clear evidence. Write this as if you will have to defend every decision in a review meeting.

E — EXAMPLE Example only: [P0] User opens login page and sees username and password fields [P0] User logs in and lands on the home page [P1] User logs out successfully (Replace with the actual steps provided below.)

P — PERSONA You are a careful, experienced automation engineer who has been in these situations before. You assume the UI is slow, elements go stale, and clicks get intercepted—and you guard against all of it.

O — OUTPUT Return only: Code Only without markdown or any notes in downloadable format

Thinking completed

// FILE: BaseTest.java
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.chrome.ChromeOptions;
import org.openqa.selenium.support.ui.WebDriverWait;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeMethod;
import java.time.Duration;

public class BaseTest {
protected static WebDriver driver;
protected WebDriverWait wait;

```
@BeforeMethod(alwaysRun = true)
public void setUp() {
    ChromeOptions options = new ChromeOptions();
    options.addArguments("--disable-notifications");
    options.addArguments("--start-maximized");
    options.setAcceptInsecureCerts(true);
    driver = new ChromeDriver(options);
    driver.manage().timeouts().implicitlyWait(Duration.ZERO);
    wait = new WebDriverWait(driver, Duration.ofSeconds(20));
    wait.pollingEvery(Duration.ofMillis(500));
    wait.ignoring(org.openqa.selenium.StaleElementReferenceException.class);
}

@AfterMethod(alwaysRun = true)
public void tearDown() {
    if (driver != null) {
        driver.quit();
    }
}
```

```java
// FILE: LoginPage.java
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;

public class LoginPage {
private final WebDriver driver;
private final WebDriverWait wait;
private static final By USERNAME_FIELD = By.cssSelector("[data-testid='username-input']");
private static final By PASSWORD_FIELD = By.cssSelector("[data-testid='password-input']");
private static final By LOGIN_BUTTON = By.cssSelector("[data-testid='login-submit-btn']");
private static final By ERROR_MESSAGE = By.cssSelector("[data-testid='auth-error-msg']");
```

```java
public LoginPage(WebDriver driver, WebDriverWait wait) {
    this.driver = driver;
    this.wait = wait;
}

public void navigateToLogin(String url) {
    driver.get(url);
    wait.until(ExpectedConditions.urlContains("/login"));
    waitUntilPageIsReady();
}

public boolean isLoginFormVisible() {
    return wait.until(ExpectedConditions.visibilityOfElementLocated(USERNAME_FIELD)).isDisplayed()
            && wait.until(ExpectedConditions.visibilityOfElementLocated(PASSWORD_FIELD)).isDisplayed();
}

public void enterUsername(String username) {
    WebElement field = wait.until(ExpectedConditions.elementToBeClickable(USERNAME_FIELD));
    field.clear();
    field.sendKeys(username);
}

public void enterPassword(String password) {
    WebElement field = wait.until(ExpectedConditions.elementToBeClickable(PASSWORD_FIELD));
    field.clear();
    field.sendKeys(password);
}

public void submitLogin() {
    WebElement btn = wait.until(ExpectedConditions.elementToBeClickable(LOGIN_BUTTON));
    clickWithRetry(btn);
}

public String getErrorMessage() {
    return wait.until(ExpectedConditions.visibilityOfElementLocated(ERROR_MESSAGE)).getText().trim();
}

private void waitUntilPageIsReady() {
    wait.until(webDriver -> ((org.openqa.selenium.JavascriptExecutor) webDriver)
            .executeScript("return document.readyState").equals("complete"));
}

private void clickWithRetry(WebElement element) {
    try {
        element.click();
    } catch (org.openqa.selenium.ElementClickInterceptedException e) {
        wait.until(ExpectedConditions.elementToBeClickable(element)).click();
    }
}
```

}

// FILE: SSOPage.java

import org.openqa.selenium.By;

import org.openqa.selenium.WebDriver;

import org.openqa.selenium.WebElement;

import org.openqa.selenium.support.ui.ExpectedConditions;

import org.openqa.selenium.support.ui.WebDriverWait;

public class SSOPage {

private final WebDriverWait wait;

private static final By SSO_IDP_BUTTON = By.cssSelector("[data-testid='sso-idp-btn']");

private static final By IDP_USERNAME = By.cssSelector("[data-testid='idp-username']");

private static final By IDP_PASSWORD = By.cssSelector("[data-testid='idp-password']");

private static final By IDP_SUBMIT = By.cssSelector("[data-testid='idp-submit']");

```
public SSOPage(WebDriverWait wait) {
    this.wait = wait;
}

public void initiateSSO() {
    WebElement btn = wait.until(ExpectedConditions.elementToBeClickable(SSO_IDP_BUTTON));
    btn.click();
    wait.until(ExpectedConditions.urlContains("idp.auth"));
}

public void authenticateWithIdP(String idpUser, String idpPass) {
    wait.until(ExpectedConditions.visibilityOfElementLocated(IDP_USERNAME)).sendKeys(idpUser);
    wait.until(ExpectedConditions.visibilityOfElementLocated(IDP_PASSWORD)).sendKeys(idpPass);
    wait.until(ExpectedConditions.elementToBeClickable(IDP_SUBMIT)).click();
}
```

}

// FILE: MFAPage.java

import org.openqa.selenium.By;

import org.openqa.selenium.support.ui.ExpectedConditions;

import org.openqa.selenium.support.ui.WebDriverWait;

public class MFAPage {

private final WebDriverWait wait;

private static final By MFA_INPUT = By.cssSelector("[data-testid='mfa-code-field']");

private static final By MFA_SUBMIT = By.cssSelector("[data-testid='mfa-verify-btn']");

private static final By MFA_ERROR = By.cssSelector("[data-testid='mfa-error-msg']");

```
public MFAPage(WebDriverWait wait) {
    this.wait = wait;
}

public void enterMfaCode(String code) {
    WebElement field = wait.until(ExpectedConditions.elementToBeClickable(MFA_INPUT));
    field.clear();
    field.sendKeys(code);
    wait.until(ExpectedConditions.elementToBeClickable(MFA_SUBMIT)).click();
}

public String getMfaError() {
    return wait.until(ExpectedConditions.visibilityOfElementLocated(MFA_ERROR)).getText().trim();
}
```

}

```
// FILE: HomePage.java
import org.openqa.selenium.By;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;

public class HomePage {
private final WebDriverWait wait;
private static final By DASHBOARD_CONTAINER = By.cssSelector("[data-testid='main-dashboard']");
private static final By WELCOME_TEXT = By.cssSelector("[data-testid='user-welcome']");
private static final By LOGOUT_BUTTON = By.cssSelector("[data-testid='logout-btn']");
```

```
public HomePage(WebDriverWait wait) {
    this.wait = wait;
}

public boolean isDashboardLoaded() {
    return wait.until(ExpectedConditions.visibilityOfElementLocated(DASHBOARD_CONTAINER)).isDisplayed()
}

public String getWelcomeMessage() {
    return wait.until(ExpectedConditions.visibilityOfElementLocated(WELCOME_TEXT)).getText().trim();
}

public void clickLogout() {
    wait.until(ExpectedConditions.elementToBeClickable(LOGOUT_BUTTON)).click();
}
```

```
}
```

// FILE: LoginFeatureTest.java import org.openqa.selenium.support.ui.ExpectedConditions; import org.testng.Assert; import org.testng.annotations.Test;

public class LoginFeatureTest extends BaseTest

@Test(priority = 1, description = "[P0] Valid standard login navigates to authenticated dashboard")

public void p0_StandardLogin_Success() {

LoginPage loginPage = new LoginPage(driver, wait);

loginPage.navigateToLogin("https://app.example.com/login");

Assert.assertTrue(loginPage.isLoginFormVisible(), "Login form fields must be visible");

loginPage.enterUsername("valid.user@example.com");

loginPage.enterPassword("ValidPass123!");

loginPage.submitLogin();

```
    wait.until(ExpectedConditions.urlContains("/dashboard"));

    HomePage homePage = new HomePage(wait);

    Assert.assertTrue(homePage.isDashboardLoaded(), "Dashboard container must render
after successful login");

    String welcome = homePage.getWelcomeMessage();

    Assert.assertTrue(welcome.contains("Welcome") || welcome.contains("Dashboard"),
"Business assertion: User must be greeted on dashboard");

}


@Test(priority = 2, description = "[P0] Invalid credentials displays explicit error without
redirect")

public void p0_StandardLogin_InvalidCredentials() {

    LoginPage loginPage = new LoginPage(driver, wait);

    loginPage.navigateToLogin("https://app.example.com/login");

    loginPage.enterUsername("invalid.user@example.com");

    loginPage.enterPassword("WrongPass456");

    loginPage.submitLogin();


    String error = loginPage.getErrorMessage();

    Assert.assertFalse(error.isEmpty(), "Error message must display for invalid credentials");

    Assert.assertTrue(error.toLowerCase().contains("invalid") ||
error.toLowerCase().contains("incorrect"),

            "Error message must be user-friendly and specific");

    Assert.assertFalse(driver.getCurrentUrl().contains("/dashboard"), "Must not redirect on
auth failure");

}


@Test(priority = 3, description = "[P0] SSO flow completes and lands on authenticated home
page")
```

```
public void p0_SSOLogin_Success() {
    LoginPage loginPage = new LoginPage(driver, wait);
    loginPage.navigateToLogin("https://app.example.com/login");

    SSOPage ssoPage = new SSOPage(wait);
    ssoPage.initiateSSO();
    ssoPage.authenticateWithIdP("sso.corporate.user", "IdpSecurePass!");

    wait.until(ExpectedConditions.urlContains("/dashboard"));
    HomePage homePage = new HomePage(wait);
    Assert.assertTrue(homePage.isDashboardLoaded(), "Post-SSO dashboard must load");
    Assert.assertTrue(homePage.getWelcomeMessage().contains("SSO") ||
homePage.getWelcomeMessage().contains("Welcome"),
            "Business assertion: SSO session must establish user context");
}

@Test(priority = 4, description = "[P0] MFA challenge validates correct code and grants
access")
public void p0_MFA_Challenge_ValidCode() {
    LoginPage loginPage = new LoginPage(driver, wait);
    loginPage.navigateToLogin("https://app.example.com/login");
    loginPage.enterUsername("mfa.required.user@example.com");
    loginPage.enterPassword("ValidMfaPass789");
    loginPage.submitLogin();

    wait.until(ExpectedConditions.urlContains("/mfa-challenge"));
    MFAPage mfaPage = new MFAPage(wait);
    mfaPage.enterMfaCode("123456");
```

```
    wait.until(ExpectedConditions.urlContains("/dashboard"));

    HomePage homePage = new HomePage(wait);

    Assert.assertTrue(homePage.isDashboardLoaded(), "Valid MFA must resolve to
dashboard");

    Assert.assertFalse(driver.getCurrentUrl().contains("/mfa"), "MFA challenge URL must clear
after verification");

}


@Test(priority = 5, description = "[P1] MFA challenge rejects incorrect code and allows
retry")

public void p1_MFA_Challenge_InvalidCode() {

    LoginPage loginPage = new LoginPage(driver, wait);

    loginPage.navigateToLogin("https://app.example.com/login");

    loginPage.enterUsername("mfa.required.user@example.com");

    loginPage.enterPassword("ValidMfaPass789");

    loginPage.submitLogin();


    wait.until(ExpectedConditions.urlContains("/mfa-challenge"));

    MFAPage mfaPage = new MFAPage(wait);

    mfaPage.enterMfaCode("000000");


    String error = mfaPage.getMfaError();

    Assert.assertFalse(error.isEmpty(), "System must reject invalid MFA code explicitly");

    Assert.assertTrue(error.toLowerCase().contains("invalid") ||
error.toLowerCase().contains("expired"),

            "MFA error must guide user correctly");

    Assert.assertTrue(driver.getCurrentUrl().contains("/mfa-challenge"), "User must remain on
MFA screen for retry");

}
```

```
@Test(priority = 6, description = "[P1] Logout clears session and redirects to login")
public void p1_Logout_SessionClear() {
    LoginPage loginPage = new LoginPage(driver, wait);
    loginPage.navigateToLogin("https://app.example.com/login");
    loginPage.enterUsername("valid.user@example.com");
    loginPage.enterPassword("ValidPass123!");
    loginPage.submitLogin();

    wait.until(ExpectedConditions.urlContains("/dashboard"));
    HomePage homePage = new HomePage(wait);
    homePage.clickLogout();

    wait.until(ExpectedConditions.urlContains("/login"));
    Assert.assertTrue(loginPage.isLoginFormVisible(), "Post-logout must present login form");
    Assert.assertFalse(driver.manage().getCookies().stream()
            .anyMatch(c -> c.getName().equalsIgnoreCase("session_token")),
            "Session cookie must be cleared on logout");
}
```